



摘要：

K3与Qwen3.8-Max接连亮相，证明3T级训练已走出“能不能”的阶段。K3通过改造注意力机制压缩长文本计算成本，用专家并行控制通信压力，并重塑深层网络的信息传递方式，配合系统工程优化，在有限算力下实现了近2.5倍的训练效率提升。

7月几天之内，中国大模型行业接连出现了两个超过2万亿参数的模型。

Kimi率先发布K3。它的总参数量2.78万亿，每个Token激活约1042亿参数，拥有896个路由专家，支持最长100万Token上下文。

随后，阿里又预览了总参数量2.4万亿的Qwen3.8-Max。

两个2T以上模型前后脚出现，说明训练一个3T级模型有了一些具体的、可复用的配方。

而这个配方，K3的技术报告都已经给了出来。

大家都觉得训练一个3T参数的模型很难，尤其是中国开源模型训练出这个大小的模型，更难。

因为中国的算力有一些瓶颈。

Kimi 没有披露 K3 主预训练使用了多少张卡、什么型号，也没有公布训练 Token 总量和 GPU 小时，但其资源区间应该与DeepSeek相似。他们手上并不缺数千张 Hopper 等效卡，但应该没有可以大规模铺开的 GB200 NVL72 或同等级超级节点。

这正是理解 K3 的起点。

训练没有门槛，效率和效果才是门槛

训练一个模型，原本可以拆成五个问题。模型、梯度和优化器状态能不能**装得下**，算力是不是能够**算得动**，数据能不能在数千张卡之间**传得动**，数据到位后 GPU 能不能持续**算得满**，花出去的计算最后能不能让模型**学得好**。

第一步是装得下。

K3 是 MoE 模型。每读入一个 Token，并不会调用全部 2.78T 参数，而是激活约16个专家 MoE，约计 104.2B 参数。

K3 的 2.78T 参数只按 BF16 保存一份权重，就需要大约 5.56TB。BF16 是训练常用的 16 位浮点格式，每个参数占 2 字节。训练时还需要梯度、优化器状态、各层产生的中间激活和通信缓冲，整个系统要管理的是几十 TB 数据。

因此理论上，2.78T的参数，只需要70多张 80GB GPU 理就能放得下，用上千张就能训练的起来。

但你得先把它们给拆开，并有效存储，还能互相连通着能汇总信息跑的下去。过去几年形成的分布式训练方法，解决的就是怎样拆。



2019 年的 Megatron-LM 给出了 Transformer 大矩阵的实用切法。2021 年，NVIDIA 又将张量、流水线和数据并行组合起来，在 3072 张 A100 上就跑通了 1T 稠密模型的训练迭代。它其实已经证明了，模型只要能够沿矩阵、网络层和训练数据等方向拆开，万亿参数就不再存在不可跨越的容量上限。

理论上，模型继续扩大，只需要增加设备和切分规模。

当然模型真正开始训练后，前向传播产生的中间结果要暂时保留，反向传播还需要梯度和优化器状态，都要有地方存。不同设备通信时，也要准备收发缓冲区。而这种计算过程中，任何一类数据在同一时刻集中出现，都可能让某张GPU显存撑不住。

因此，一般的训练体系中还得有一个统一调度，防止存储「冒顶」现象的出现。比如 K3 通过 Unified Activation Manager，也就是统一激活管理器，把它们整合成张量级的存储调度。每个反向传播需要的中间结果，都可以由这个管理器动态决定是留在GPU、还是转到CPU的存储去，是压缩成FP8，还是干脆不保存，等反向传播时重新计算。



装得下

统一激活管理器 (UAM)

先把 2.78T 装得下

总参数
2.78T

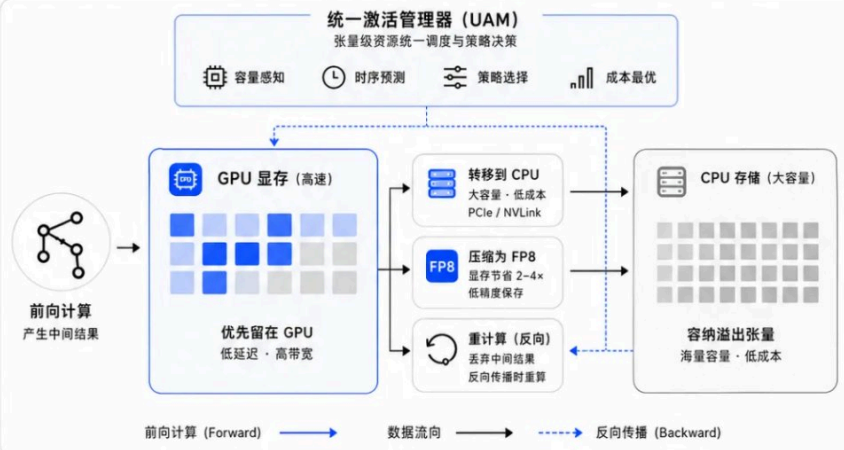
激活参数
104.2B

上下文
1M

- 训练时不仅有权重，还有梯度、优化器状态、中间激活和通信缓冲。
- UAM 统一调度这些张量级资源，避免某时刻局部“冒顶”。
- 中间结果可按需要留在 GPU、转移到 CPU、压缩成 FP8，或在反向传播时重计算。

统一激活管理器 (UAM)
张量级资源统一调度与策略决策

容量感知 时序预测 策略选择 成本最优



前向计算 (Forward) → 数据流向 → 反向传播 (Backward)

关键点

能切、能训不炸，是后面一切优化的前提。

它像一个统一仓储系统。马上要用、重新制造又昂贵的零件放在线边、暂时不用的送去外仓。体积大的压缩存放，便宜易造的零件不留库存，用到时再生产。

这种细粒度调度则就负责避免训练运行到某一层、某个批次时突然局部爆仓。

到这一步，就是能切，能训不炸了。

但能切不等于能高效训练。模型切得越散，原本发生在一张GPU内部的数据交换，就越多地变成跨卡通信。而且，设备增加以后，专家负载、流水线等待和故障概率也会同步上升。

因此，能切以后，大模型竞争的核心从「参数能不能放下」，转向四个问题。**怎么才能算的更快，卡间需要传多少数据，GPU有多少时间真正用于计算，以及每单位FLOPs能换来多少模型质量。**

能在有限算力中高效训练出来，更大参数这事儿才能成立。

而过往的经验中，大家都也都是主要在这四个点上下功夫。招式也几乎差不多。比如到了 DeepSeek-V3 这一代，大规模 MoE 的主要优化方向已经基本定型。

招数可以分为几大方向。

算得动，主要靠**修改注意力机制**，减少长上下文的存储与计算。DeepSeek从MLA开始选择了稀疏路径，Minimax和Kimi选择了线性。

传得了，一来是控制单个 Token 的激活成本。总参数继续扩大，但每个 Token 实际调用的参数、产生的计算和通信不能同步增长。这就是MoE的作用。

二是把训练精度压小，比如用FP8，乃至FP4的数据训练。

三来是疏通堵点，比如平衡专家负载，避免少数热门专家堵住整个集群。DeepSeek 使用无辅助损失负载均衡。

算的满，主要依靠重排并行与流水线，让通信和计算重叠，挤掉 GPU 空等的气泡。

最终，依靠压缩稀疏处理和高效运用GPU，模型训连的整体速度自然快。

但学得快还有另一层意思，就是提高每单位训练计算的学习回报。这可以通过**优化器（Muon）**和**残差流改造**。

这套框架和方法点，后来的 DeepSeek-V4 和 Kimi K3 这次在模型上的三项架构改造基本上是完整的进行了继承，只是增加了细化。

预训练主战场			
三大超大模型架构对比			
DeepSeek-V3 / DeepSeek-V4 / Kimi K3			
对比维度	DeepSeek-V3	DeepSeek-V4	Kimi K3
	总参数 671B 激活参数 37B	总参数 1.6T 激活参数 49B	总参数 2.78T 激活参数 104.2B
算得动	DeepSeekMoE让671B总参数只激活37B；MLA压缩注意力表示，但仍是全局注意力。	1.6T总参数只激活49B；CSA/HCA通过序列压缩和稀疏查询，降低百万上下文计算。	2.78T总参数激活104.2B；KDA让大部分长上下文注意力近似线性增长；LatentMoE压缩专家计算空间。
传得动	MLA压窄每个Token的Key/Value；限制专家路由范围；无辅助损失负载均衡减少专家拥堵。	CSA/HCA减少需要保存和查询的历史；低精度表示减少传输字节；两阶段上下文并行。	LatentMoE把专家输入从7168维压到3584维；KDA上下文并行交换固定状态；Quantile Balancing和MoonEP疏通专家拥堵。
算得满	DualPipe让前向、反向和专家通信重叠；FP8提高Tensor Core效率；专用all-to-all和专家矩阵计算。	融合MoE Kernel，让计算、通信和显存访问重叠；细粒度重叠计算与低精度执行。	FlashKDA提高前向状态计算效率；MoonEP提供固定负载和静态状态；零拷贝复制、统一流控管理与通信重叠。
学得好	细粒度和共享专家提高参数利用率；MTP增加每个Token的训练信号；无辅助损失均衡避免干扰主目标。	mHC改善深度信息流；Muon加快收敛并稳定训练；更大的数据配方。	AttrRes提高新增深度的利用率；Per-Head Muon平衡不同注意力头；RMSNorm、SiLU-GLU和全静KDA输出门提高稳定性和表达能力。
总结	V3在工程体系上实现高效稳健的规模化；V4在长上下文与系统协同上更进一步；K3在长上下文近似线性扩展与专家系统上走得更远，三者各有侧重，共同推动大模型能力边界向前。		

但K3面对了一些新挑战。

从1T到2.8T，新增参数必须以某种方式进入模型的实际计算，而且得让它尽可能有用。不然就白白浪费了。K3因此同时扩大了宽度、深度和长度。

变宽最直接地产生参数。K3把路由专家从384个增加到896个，让模型拥有更大的专业参数库。同时把每Token激活专家从8个增加到16个，使新增容量不只是静静躺在模型里，而能更多地参与一次计算。

变深增加每个Token接受加工的次数。K3从61层增加到93层，使信息能够经过更多轮变换。只增加专家，相当于扩建图书馆，却没有增加研究员处理材料的步骤。增加深度，才让模型能够把更大的参数容量组织成更复杂的表示和推理过程。

变长则扩大模型需要处理的问题范围。上下文从128K增加到100万Token，并不会直接增加参数，但它让更大的模型在训练中面对更长的依赖、更复杂的材料和更长期的任务。否则模型虽然拥有2.8T参数，训练时仍只处理较短的信息片段，新增容量未必能转化成长程能力。

100万Token会让全局注意力计算急剧增长，算不过来。K2还能用的MLA必须得换成线性了。

更多专家和一次用一倍的专家，则会增加通讯负担和负载不均风险。

93层会增加学习的效果压力，残差在越来越深的层里迷失，就学不会了。

因为当前算力本身的限制，算的过来和传的动是最大的卡点。因此和DeepSeek一样，在稀疏和均衡上，Kimi先下了大力。

第一关：KDA 解决算得动

K3 把上下文从 128K 拉到 1M，网络又从K2时代的 61 层加深到 93 层。如果 93 层全部沿用 MLA，序列长度增加约八倍后，算力的增加完全没法处理。

这是因为全注意力处理 100 万个 Token 时，会把所有词全部摆开，让每个新词都能看到之前的所有词。代价是 $100 \text{ 万} \times 100 \text{ 万} = 1 \text{ 万亿次}$ 巨量计算。

而线性注意力的核心目标，它不再保留全部历史，而是把读过的内容不断压进一个固定大小的状态矩阵。**是把全注意力那个存储了所有历史信息的的N×N的巨大矩阵，压缩成一个小巧的、可以持续更新的记忆胶囊。**

它维护一个固定大小的状态向量（比如 128 维），每来一个新词就更新这个状态。写入新信息，部分保留旧信息，部分遗忘。无论序列多长，只存储这个固定大小的状态，计算量降到 $O(N)$ ，内存变成恒定。

材料从一万字增加到一百万字，这个胶囊还是一样的大小。

除了算得快以外，线性注意力也从根本上改变了长上下文的成本曲线。由于每一步面对的状态大小基本不变，序列增加一倍，注意力计算通常只增加一倍，而不是四倍。

但线性注意力本质上，是在用“有损压缩”换取效率。**这种有损压缩带来的核心问题是精确检索困难**。当你需要从 100 万个 Token 中精确找回第 3 万个位置的某个关键信息时，那个被压缩了 97 万次的状态，已经很难给你准确答案了。

想要让它能减少这种压缩中的损失，就得让它更好的选择记忆什么和忘掉什么。

这就靠的是 Kimi Linear 架构的核心，KDA (Kimi Delta Attention)注意力机制。

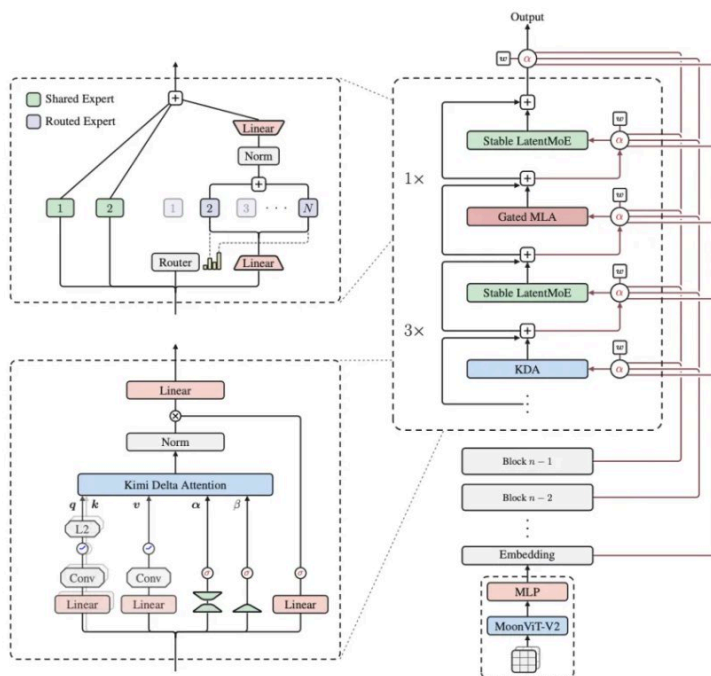


Figure 2: The Kimi K3 architecture, organized around token, channel, and layer mixing, with a native vision pathway at the input. Each block contains three Kimi Delta Attention (KDA) layers followed by one Gated MLA layer, with each attention layer paired with a Stable LatentMoE feed-forward network. Attention Residuals (AttnRes) use learned pseudo-queries (w) to derive attention weights (α) over the embedding and preceding block outputs, enabling selective information flow across depth. **Top left:** the Stable LatentMoE module with shared and routed experts. **Bottom left:** the KDA module. **Bottom right:** the native vision pathway.

(K3的架构整体)

一个token进入后要经历生成控制信号 → 遗忘旧状态 → 按更新强度纠错写入 → 读取新状态 → 输出门筛选 → 送入下一层的全流程。

算得动

KDA

让百万上下文算得动

KDA 层

69



MLA 层

24

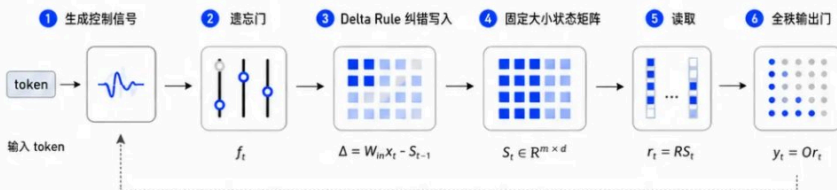


混合结构

3:1



KDA 工作流程



3:1 混合结构（重复堆叠）



- 1 KDA 把历史信息压进固定大小状态矩阵，长上下文成本近似线性增长。
- 2 通道级遗忘门支持不同通道以不同速度衰减；单步保留率最低约 0.67%，避免数值爆炸。
- 3 Delta Rule 先读后改，全秩输出门更细粒度筛选读出内容。
- 4 K3 采用 3:1 混合结构：连续 3 层 KDA，插入 1 层 MLA；最终为 69 层 KDA、24 层 MLA。



关键点

KDA 让大部分长上下文注意力的成本从平方增长转向近似线性增长。

先过的是通道级的遗忘门，它确定哪个状态可以被遗忘。

为了保证更精细地管理有限记忆，KDA在注意力头中引入了「通道级别」的细粒度门控机制。在遗忘门这里，就是可以保证状态矩阵的不同方向可以用不同速度衰减。比如实体关系可能需要长期保留，局部措辞和临时格式则可以更快淡化。这样关键结论记得久，临时提醒会加速被忘掉。

Kimi Linear允许模型在一步之内，把某个记忆通道几乎完全清空。这样看起来很灵活，却会给并行计算制造麻烦。KDA为了同时计算一小段Token，需要先把不同位置经历的衰减换算到同一尺度。如果某段信息已经被淡化到接近零，计算机就必须用一个极大的倍数把它重新放大，数字很容易超出BF16能够表示的范围。

同一个小块内部的部分位置关系，也因此无法使用GPU最擅长的整块矩阵乘法，只能逐对处理。

K3给遗忘踩了一脚刹车，单步保留率最低约为0.67%，不允许任何通道在一步内彻底归零。这不会让模型忘不掉，因为连续多步衰减后，信息仍会迅速变得极小。它只是避免计算过程中突然出现过于极端的数字，KDA也因而跑得更稳、更快。

下一步纠错写入的更新的规则，KDA用了 Delta Rule，也就是增量纠错规则。模型写入新内容前，会先根据当前 Key 从状态中读一次，比较现有记忆与新 Value 的差异，再针对误差做修改。靠着它，线性注意力可以记得更准。

KDA 完成修改后并不直接把token送到后面的神经网络中，这里还有一道输出门。它负责决定候选信息里的哪些特征可以进入下一层。

可以把它想成一间录音室。模型的Query操作先从不改好的资料库里找回许多声部，输出门再像调音台一样，压低这次任务无关的通道，放大有用信号。

它不会修改笔记里长期保存了什么，只控制这一次读出的内容怎样影响后续计算。

K3 为了进一步保证信息更有效的输出，把原始Linear的低秩输出门改成了全秩输出，让当前Token 可以更细地调节各个输出通道。它增加了一点计算，却让固定状态的读取更灵活。

即使如此，固定状态一定会损失细节。它不可能无损装下一百万 Token 的每个字符。K3 因此没有完全放弃全局 Attention，而是沿用 Kimi Linear 的 3：1 混合结构：连续三层 KDA 后插入一层 MLA。K3 最终有 69 层 KDA 和 24 层 MLA，并在模型末尾用 MLA 收尾。

固定状态一定会损失细节。它不可能无损装下一百万 Token 的每个字符。K3 因此没有完全放弃全局 Attention，而是沿用 Kimi Linear 的 3：1 混合结构：连续三层 KDA 后插入一层 MLA。K3 最终有 69 层 KDA 和 24 层 MLA，并在模型末尾用 MLA 收尾。

MLA负责定期重新查看全部历史Token，弥补线性注意力难以精确找回原文细节的问题。

K3的MLA不再使用显式位置编码，前后顺序和远近信息主要由KDA提供，因此扩展上下文时不必重新调整RoPE，计算量也有明显减少。

多数注意力层的计算变成近似线性增长，大幅减少，这样大幅减少了算力需求，算的动了。

第二关，压缩和防堵车系统解决传得了

所有分布式训练方法都在做同一笔交换：用通信换显存。

一个公司，把机器、楼层、专业车间和账本分散到不同园区，确实解决了单个园区装不下的问题；但从此以后，半成品、生产记录和结算信息都必须在园区之间运输。

因此并行之后，信息传递就大大增大了。

而并行的方法很多，又分成张量并行把同一层的巨大矩阵切给多张GPU。**流水线并行**把不同网络层交给不同设备组。每张卡只需要保存部分层及其训练状态。**数据并行**让多个设备组同时处理不同训练样本，它换来的是更高的Token吞吐。

上下文并行则是把一条长序列沿Token方向分给多张GPU。它换来的是每张卡只保存和处理一部分上下文序列的激活。

专家并行则是MoE独有的特色，它把不同专家固定在不同GPU上，换来的是每张卡只保存专家池的一部分，而不是全部896个专家。但代价是专家权重虽然不动，Token却必须移动，一组训练数据得在被激活的专家间来回跑。

LatentMoE，压缩更多专家带来的膨胀

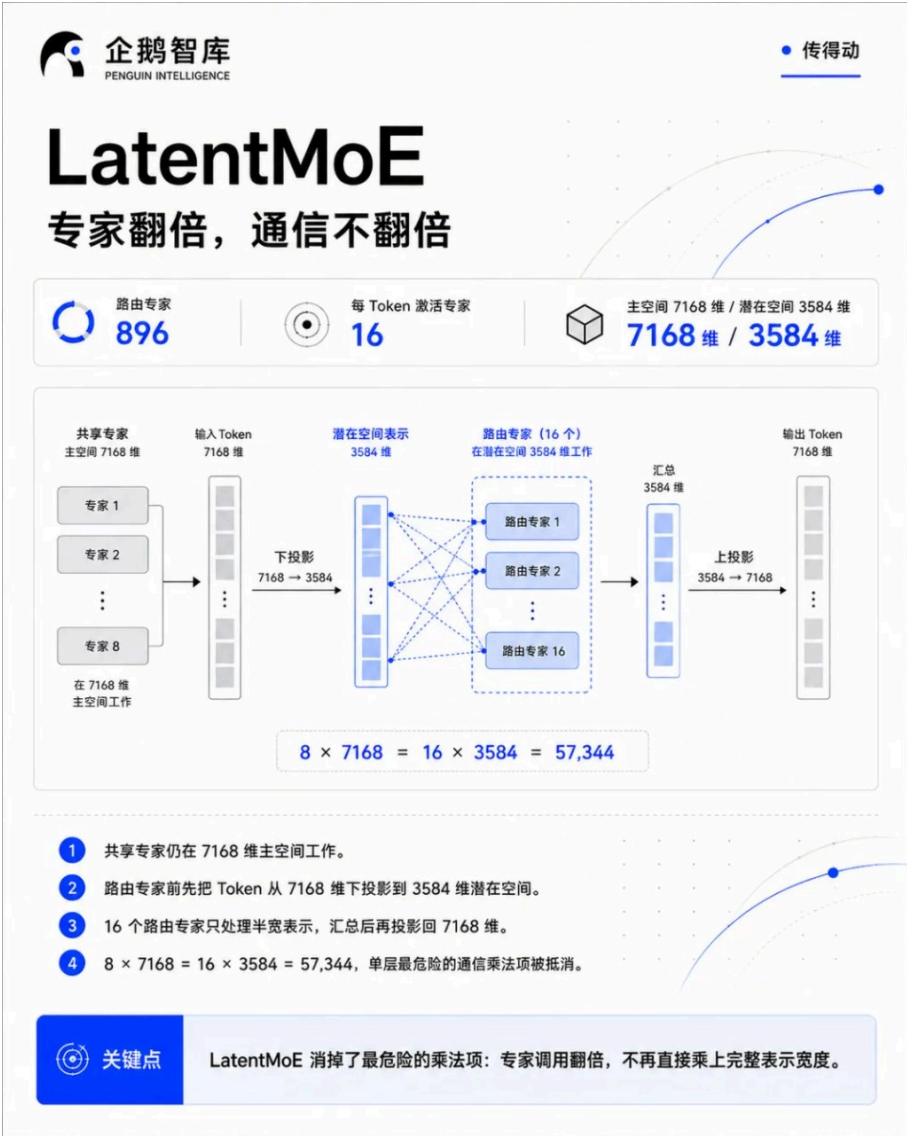
虽然有这么多并行模式，但这次承担参数增长的主要是专家总数和激活专家数量增多。



MoE模型，并不会让每个Token都经过全部参数。每个Token只选择其中几个，而这一次训练中主要的信息传递就发生在 token的激活被分发被选中的专家，专家完成计算后，再将多路结果返回并汇总。

而K3把每Token激活专家从8个增加到16个，压力一下子翻了一倍，专家并行自然成为「传得动」的主要难题。

针对这一问题，K3采用了Nvida在1月提出的 LatentMoE 技术，可译作潜在空间混合专家，就是把需要分给专家的参数压缩一倍。



K3 的两个共享专家仍在 7168 维主空间工作，所有 Token 都会经过它们。路由专家收到数据前，系统先用一次下投影，把 Token 从 7168 维压缩到 3584 维潜在空间。而16 个路由专家只处理这份半宽表示，结果汇总后，再投影回 7168 维。

未经压缩的K2需要处理的维度是8 个专家 × 7168 维 = 57,344，经过LatentMoE 压缩后，K3则是16 个专家 × 3584 维 = 57,344，刚好完全一致。

靠着这一手，专家数量增加了，需要传递的材料总量并没变厚。

这不等于 K3 的整体通信和 K2 相同。但K3从61层增加到93层，MoE层数量也从约60层增至92层，深度还是增加了的。但K3从61层增加到93层，Token需要经历更多轮专家分发与结果回收。

但 LatentMoE 消掉的是最危险的一次乘法，专家调用翻倍，不再直接乘上完整表示宽度。

KDA 上下文并行，让上下文的交接变薄

另一个大幅增加的传输压力，来自于上下文并行。原来不到1M的上下文并不需要很多并行，但 K3是需要的。

为应对这一点，KDA 在卡间通信上做了更多优化，这就是KDA Context Parallelism，即 **KDA 上下文并行**。



这样，模型计算需要交换的对象就变得大小固定，不再随本段 Token 数量一起增长。这在长上下文训练情况下大幅同样大幅降低了跨GPU通讯的压力。

Quantile Balancing，解决线路堵车问题

平均通信量被压下来，MoE的网络仍然可能会被堵死。

路由器不会自然把 Token 均匀分给 896 个专家。某批代码数据可能让一组专家特别热门，另一组专家几乎无人访问。如果几个热门专家恰好在同一张 GPU 上，这张卡就会收到大量 Token。其他卡已经算完，也只能等它。

DeepSeek-V3之前，MoE通常会在主训练目标之外增加一个负载均衡损失，迫使Token平均选择专家。但这个额外目标可能干扰模型学习，路由器为了平均分配任务，不一定把Token交给最合适的专家。

DeepSeek-V3改成给每个专家增加一个路由偏置。热门专家的偏置降低，冷门专家的偏置提高。这个偏置只影响专家能否进入Top-k，不改变专家入选后的混合权重，因此不会通过辅助损失直接干扰模型主目标。

它的更新方法很像手动调节水龙头。专家过载，就把偏置降低固定的一小步。专家负载不足，就把偏置提高固定的一小步。

DeepSeek-V3只有256个路由专家，已经可以用这种办法维持大致均衡。

K3扩大到896个路由专家以后，固定步长的问题更加明显。Kimi因此提出Quantile Balancing，中文可以译为**分位数负载均衡**。

比起拧水龙头，它直接开始看水表。查看当前全局Batch中的路由分数分布，计算每个专家应该设置多高的录取线，才能大约收到目标数量的Token，一次性整体调节，让长期负载靠近目标水平。



Quantile Balancing

先看分位数，再调录取线



路由专家
896



目标
长期负载均衡



作用阶段
全局 Batch

- 1 路由器不会自然把 Token 均匀分给 896 个专家。
- 2 系统统计全局 Batch 的路由分数分布，估计每个专家应设置多高的录取线，才能接近目标 Token 数。
- 3 它一次性整体调节长期流量，比固定步长偏置更适合超大规模专家系统。
- 4 Quantile Balancing 解决的是长期流量分布，而不是瞬时堵车。

路由分数分布与动态录取线（按分位数）

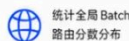
调整前：分布不均，部分专家过载或欠载
—— 路由分数分布 --- 固定录取线



按分位数估计并调整录取线
使每个专家接近目标 Token 数

调整后：长期负载均衡，各专家接近目标 Token 数

—— 路由分数分布 --- 动态录取线（按分位数）



统计全局 Batch
路由分数分布



计算分位数
确定各专家录取线



执行动态录取线
实现长期负载均衡



关键点

在超大规模专家系统里，先把长期流量调顺，后面的实时调度才有基础。

到这里，「传得动」的分工就清楚了。KDA 减负长上下文并行带来的流量，LatentMoE 压缩专家沟通带来的流量，减负。而Quantile Balancing 调整长期流量。

第三关，集体底座调整，解决算得满

训练系统常说峰值 FLOPs。FLOPs 是浮点运算次数，可以粗略表示 GPU 做矩阵计算的能力。把几千张卡的峰值 FLOPs 相加，得到的只是理论上限。

真实训练速度取决于这些计算单元有多少时间在干活。

一张 GPU 可能因为等待远端 Token 而空着，也可能已经拿到数据，却只分到一个很小的矩阵。前一种是通信饥饿，后一种是任务颗粒太碎。两者都会让硬件利用率远低于卡的标称性能。

可以把 GPU 想成餐厅后厨。厨师很多不代表出菜快。食材没送到，厨师只能等。

Megatron-Core 提供了大模型工厂的通用排产框架，但 K3 的三种新架构改变了原料形状。KDA 带来递归状态，AttnRes 增加跨 Block 表示，LatentMoE 又产生接近一千个动态专家。

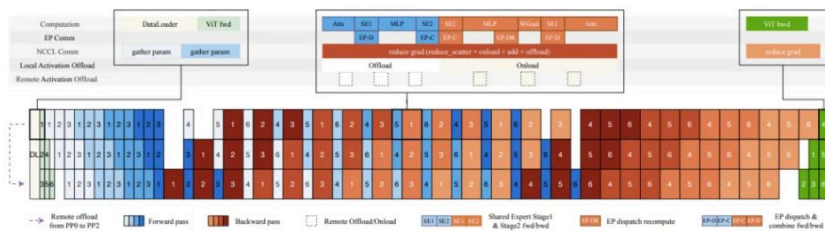


Figure 11: Computation, communication and offloading overlapped in different PP phases.

(Kimi K3的总排程图)

所以一番改造，才能让模型算得满。

MoonEP，让GPU里的专家都有活儿干

比如说Quantile Balancing根据整个全局Batch的路由统计，计算下一步应该使用的专家偏置。

它像根据全市一天的平均车流调整红绿灯，却无法预先知道下一分钟会不会突然来一大队40辆联排婚车。

而一个全局Batch通常由许多Micro-batch组成。假设整个Batch同时包含代码、数学和普通文本，汇总起来，每个专家收到的Token可能很均匀。但其中某个Micro-batch可能恰好以代码为主，大量Token会同时选择擅长代码的几名专家。

后面的数学和文本Micro-batch能够把全天平均数拉平，却无法消除当前这一步造成的部分GPU的空耗。

MoonEP 就是在系统层处理已经发生的拥堵。MoonEP在Quantile Balancing的路由形成之后、真正堵车之前，读取当前 Micro-batch 的真实分配情况。

发现某些 GPU 上的专家过热后，它会在其他 GPU 上临时复制这些专家，并把部分 Token 分流给副本。由此保证每个专家并行设备（通常对应一张 GPU）最终收到完全相同数量的Token。

如果 Quantile Balancing 是调整各收费站的长期准入政策，MoonEP 就是实时交通指挥。某个收费站前已经排起长队，系统会在空闲车道临时开一个相同窗口，把后续车辆导过去。

MoonEP

系统层实时疏通堵车

作用阶段
Micro-batch目标
每个 EP 设备等量接收 TokenZero-copy
Communication

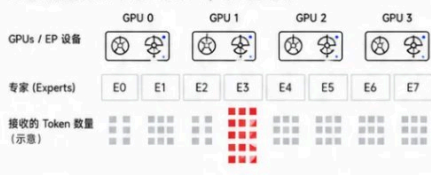
1 某些 Micro-batch 可能让少数专家突然过热。

2 MoonEP 在真实分配已经形成后，临时在其他 GPU 上复制过热专家，把部分 Token 分流给副本。

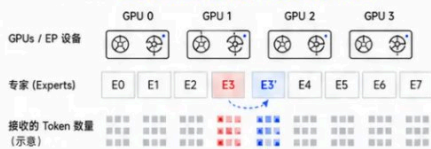
3 由此保证每个专家并行设备最终收到完全相同数量的 Token。

4 Zero-copy Communication 让 Token 直接落到按专家分组的目标位置，减少显存内额外搬运。

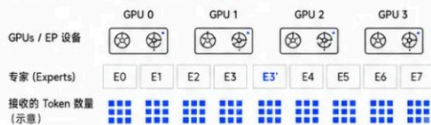
1. 过热发生：某个 Micro-batch 中，专家 E3 过载



2. MoonEP 疏通：临时复制过热专家 E3，并分流部分 Token



3. 负载均衡：每个专家并行设备收到等量 Token



Zero-copy
Communication

Token 直接落到按专家分组的目标位置 (专家缓冲区)

来自其他 GPUs 的 Token



E0

E1

...

E7

直接落盘，无需中转拷贝



关键点

MoonEP 不减少总计算，但避免同一批信息堵在局部设备上。

完全配平会带来几项连锁收益。每张卡的专家工作量一致，训练不再等待最忙设备。每个设备接收的 Token 总数固定，通信缓冲区可以按固定大小准备。

MoonEP 还使用 Zero-copy Communication，即**零额外复制通信**。规划器提前知道每个 Token 在远端属于哪个专家，于是数据可以直接落到已经按专家分组的目标位置，不必在显存里搬运一次。

专家调度器，把零碎专家任务拼成 GPU 喜欢的大矩阵

经过上面的MoonEP 和Quantile Balancing，堵车问题就不会出现了。

但专家负载即使在 GPU 之间完全配平，同一张 GPU 内部的专家也未必同样忙。一个专家收到 200 个 Token，另一个只收到 5 个。如果系统按固定顺序逐个计算，部分计算单元处理大任务，其他计算单元很快结束，出现的情况就是算不满。

K3 在此基础上又增加了 Workload-aware Scheduler，即**负载感知的专家计算调度器**。

它会在计算开始前读取当前各专家收到的Token数，再根据硬件成本模型调整任务安排。热门专家的计算得到更多处理资源，小专家则穿插执行。总计算量没有减少，但计算单元的空等时间缩短了。

负载感知专家调度器

把零碎任务拼成 GPU 喜欢的大矩阵



作用范围
单张 GPU 内部

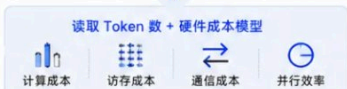


输入
各专家 Token 数



目标
减少计算单元空等

调度前：负载不均，计算单元空等多



重新安排
执行顺序
和资源分配

调度后：负载重排，GPU 内部更接近满载



- 1 GPU 之间即使完全配平，同一张 GPU 内部的专家也可能一忙一闲。
- 2 调度器读取当前各专家的 Token 数，并结合硬件成本模型重新安排执行顺序和资源。
- 3 热门专家获得更多处理资源，小专家穿插执行，减少计算单元空等。
- 4 总计算量没有减少，但 GPU 内部更接近满载。



关键点

先配平，再重排，才能把专家计算真正压成 GPU 擅长的大矩阵。

这就像物流中心已经保证每个仓库收到 1000 箱货，但仓库内部各货架的箱数仍不同。调度器要根据这一批真实货物重新安排叉车，不能永远按固定货架顺序搬。

FlashKDA，让线性计算也能沾上并行的光

KDA 让历史状态变小，却引入了 GPU 不喜欢的先后依赖。

线性注意力下，一个序列块的出口状态要传给下一个块，后一个块才能得到准确结果。若系统先并行计算块内 Token，再停下来串行传播状态，GPU 会在传播阶段闲置。

FlashKDA 是 Kimi 为 KDA 编写的两个专用 Kernel，也就是一张交给 GPU 如何去计算的工单。

他会先把每段 Token 会怎样修改记忆，预先整理成一张「更新卡」，包括遗忘门计算、Query 和 Key 归一化、衰减处理，以及块内纠错所需的小矩阵构造。这些更新卡互不依赖，可以同时计算。

等全部准备好后，模型再带着记忆按顺序经过每张卡，完成状态更新、输出读取和最终状态累积。

FlashKDA

把串行依赖压到最后



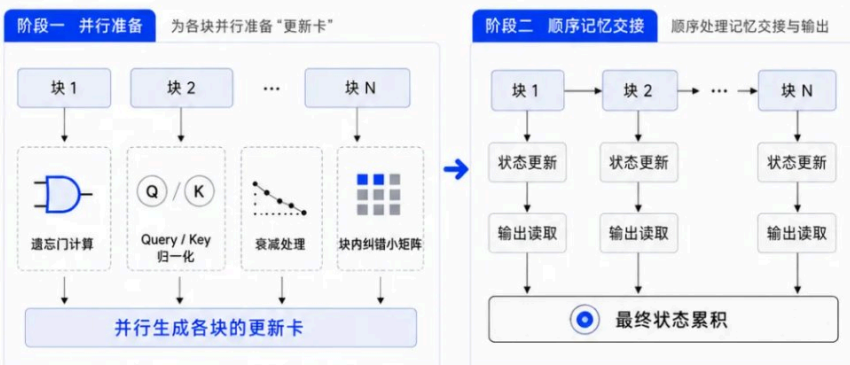
专用 Kernel



阶段一 并行准备



阶段二 顺序记忆交接



- KDA 有递归状态，后一个块要等前一个块的出口状态。
- FlashKDA 先并行准备“更新卡”：遗忘门计算、Query / Key 归一化、衰减处理和块内纠错小矩阵。
- 之后只把最后的记忆交接放进顺序流程，减少 GPU 因等待状态传播而空转。
- 它不是消灭前后依赖，而是把可并行部分提前拆出来。



关键点

线性注意力省下来的状态，
只有经过 **FlashKDA** 才会真正变成训练吞吐。

它没有消除KDA的前后依赖，而是把大部分可以并行的准备工作从依赖链中拆了出来，只把最后的记忆交接留在顺序流程里。这样，GPU不会每处理一小段Token就停下来等待一次状态更新。

KDA 省下的状态，只有经过 FlashKDA、KCP 和数值参数化改造，才会真正变成训练吞吐。否则模型少搬了数据，却可能在递归上把时间重新花回去。

这些方法听起来分散，实际都在解决同一件事：不要让所有数据在同一时间出现在同一张卡上，也不要让搬运成为一段纯等待。让GPU尽可能的全跑满。

第四关，残差优化，影响学得快

都算的满了，还不能证明 2.8T 值得训练。

K3 最重要的训练效率指标是 Scaling Efficiency，中文可以译作**规模扩展效率**。它问的不是单张卡峰值有多高，而是达到同一个验证 Loss，整场训练总共需要多少 FLOPs。

可以把它理解成模型的燃油效率。前面算得满，解决了发动机能转多快，而Scaling Efficiency 决定烧掉同样一桶油能走多远，也就是影响达到目标质量需要支付多少总计算。

K3 报告称，新架构、数据和训练配方共同带来约 2.5 倍整体 Scaling Efficiency。也就是说，如果 K2 路线达到某个效果需要1的计算量，K3 模型家族达到这个效果，大约只需要0.4的计算量。

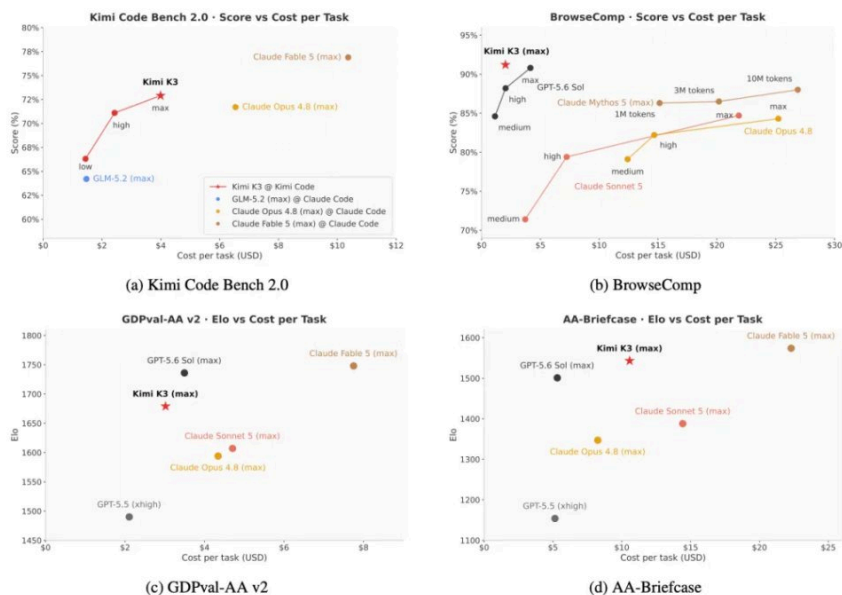


Figure 13: Score vs. per-task inference cost on Kimi Code Bench 2.0, BrowseComp, GDPval-AA v2, and AA-Briefcase. Kimi K3 is marked with a star.

前面提到的很多工作，其实已经大幅增加了K3的Scaling Efficiency。比如 Kimi Linear，在它的论文里把全 MLA 换成K3用的这种 3 : 1 的 KDA—MLA 混合结构，就可以带来1.16 倍计算最优效率。

下面要介绍的 AttnRes 其论文中称，能带来1.25倍的Scaling Efficiency。

两项合在一起，能解释大概50%左右的效率提升。

而 Stable LatentMoE 的压缩功力，也可以给它带来一定的收益。

剩下的部分，则来自于其余架构、数据和训练配方，以及它们之间的协同作用。

AttnRes，用分层残差穿透深层网络

从 2017 年至今，主流 Transformer 的学习机制主要是残差连接。每一层完成计算后，把新产生的结果直接加到原来的隐藏状态上：**新的状态 = 原来的状态 + 本层产生的信息**。

这个设计非常简单，却解决了深层网络最棘手的训练问题。即使某一层暂时什么都没有学到，信息和梯度也可以沿着残差通道继续向前传播。没有残差连接，今天上百层的大模型很可能根本训练不起来。

问题在于，残差连接只负责把信息传下去，却不负责判断哪些信息值得保留。

把残差公式逐层展开，就会发现最终的隐藏状态，本质上是许多历史层输出的持续累加。浅层提取的词法特征、中层形成的句法关系、深层产生的推理表示，都被塞进同一条残差流中。

无论某一层提供的是关键线索、重复信息还是噪声，它们进入主干道时使用的都是近似相同的加法规则。

层数较少时，这种粗糙的方式尚可工作。但随着模型越来越深，残差流中累计的历史信号不断增大，每一层新产生的变化反而显得越来越微弱。

到了90多层这样的位置，一层即使进行了大量矩阵计算，最终对隐藏状态造成的改变也可能非常有限。

输入和输出越来越相似，这一层便逐渐接近恒等映射。

这就是所谓的「深度诅咒」，即模型增加了层数和参数，却没有获得相同比例的有效深度。

一些研究发现，在32层模型中，接近44%的层可能只产生非常有限的有效变换。模型看起来变深了，真正参与表示学习的层数却增长得很慢。

Transformer 的每一层都会在输入表示上计算一个新变化，再通过残差连接把变化加回主干。残差连接让深层网络更容易训练，因为信息和梯度可以沿一条直接通道前后传播。

问题在于，普通残差把此前所有加工结果不断混进同一份状态。

K3 所采用的 Attention Residuals，也就是 AttnRes 就是要终结这种「深度诅咒」，让残差的学习变得更有效率。

既然注意力可以决定一个 Token 应该读取哪些 Token，为什么不能让它决定当前层应该读取哪些历史层？

在标准残差连接中，历史信息通过固定加法进入当前状态。AttnRes 则把不同层产生的表示视为一组候选信息。

模型进入新一层时，会根据当前状态生成查询，再通过注意力计算判断，此刻更需要浅层保存的局部特征、中层形成的结构信息，还是最近几层产生的高阶语义。

换句话说，标准残差连接只有累加，AttnRes 增加了检索和选择。

深层不再只能拿到一杯经过几十次混合的「信息果汁」，而是可以回到历史层中，直接选择当前真正需要的那部分表示。层间信息传递由固定的数据通道，变成了一个可学习的动态路由系统。



AttnRes

让 93 层网络真正学得动



网络深度
93 层



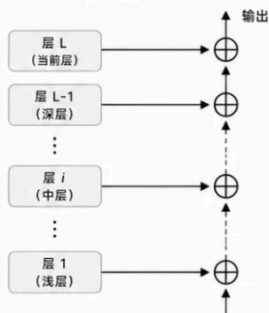
Block AttnRes
约 **8** 个 Block



Scaling Efficiency
约 **1.25x**

标准残差 (Residual)

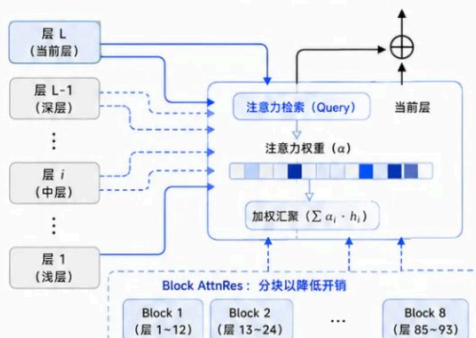
只会不断累加历史层输出



逐层累加，深层趋近恒等映射

AttnRes (注意力残差)

让当前层用注意力主动选择历史层表示



可选择浅层细节 / 中层结构 / 最近高阶语义

- 1 标准残差只会不断累加历史层输出，深层容易接近恒等映射。
- 2 AttnRes 把历史层表示视为候选信息，让当前层用注意力主动选择浅层细节、中层结构或最近高阶语义。
- 3 它让深层不再只拿到一杯混合后的“信息果汁”，而是能够直接回看需要的历史层。
- 4 Block AttnRes 把连续层划成若干块，保留跨层检索能力，同时控制额外开销。



关键点

固定残差通道，被升级成了可学习的纵向信息路由。

这种改变还带来一个容易被忽略的能力：遗忘。

传统残差连接几乎只进不出。早期信息一旦进入残差流，就会继续参与后面的计算。AttnRes 则可以降低无用层的权重，让模型不必始终携带全部历史信息。它解决的因此不只是浅层信号被稀释的问题，也是在帮助模型控制残差流中的信息密度。

当然，直接让每一层关注此前所有层，会带来额外的计算和存储成本。模型越深，需要参与注意力计算的历史状态就越多。

为此，Kimi 又提出了 Block AttnRes，把连续层划分为若干块。这样既保留了深度检索能力，又避免计算量随着层数迅速膨胀。K3中，93 层被组织成大约 8 个 Block，每个 Block 约 12 层。

靠着这种新的纵向注意力系统，AttnRes 能更有效的使用残差，让 93 层神经网络这样的深度下，更容易产生有效表示。

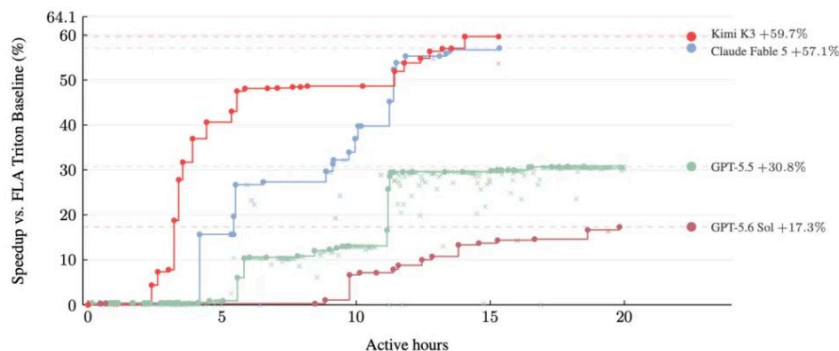


Figure 14: Case study: GPU kernel optimization on AttnRes.

后层可以直接调用浅层细节和中层关系，不必完全依赖一份高度混合的残差状态。其中的损失信号也获得了更直接通往早期阶段表示的路径。

AttnRes 论文的实验显示，这项改动的收益能随模型规模保持，并在 48B 总参数、3B 激活参数的 Kimi Linear 模型上获得了约 1.25 倍的 Scaling Efficiency。

学得快的前提，是数值不能中途爆掉

3T 训练运行数周乃至数月，偶尔出现一次极端激活都可能造成昂贵损失。Scaling Efficiency 再好，训练频繁溢出或回滚也没有意义。

模型需要一整套稳定性配方。

企鹅智库
PENGUIN INTELLIGENCE

学得好

稳定性配方

把规模扩展效率兑现出来

RMSNorm

SiTU-GLU

Per-Head Muon

KDA 衰减下限

1 RMSNorm: 稳定专家汇总后的表示尺度

2 SiTU-GLU: 用平滑上限压住极端值

3 Per-Head Muon: 避免单个注意力头主导整块投影矩阵

4 KDA 衰减下限: 减少异常数值, 兼顾稳定与效率

提升长程训练稳定性

1 RMSNorm 先把专家汇总后的表示尺度拉回稳定范围。

2 SiTU-GLU 用平滑上限压住极端值。

3 Per-Head Muon 避免单个注意力头主导整块投影矩阵。

4 KDA 的衰减下限与其他数值参数化一起，减少异常值、回滚和训练中断。

关键点

小模型上的理论效率，只有在 2.8T 级训练里足够稳定，才有实际价值。

K3 在专家汇总后加入 RMSNorm，也就是**均方根归一化**，先把潜在表示的整体尺度拉回稳定范围；又用 SiTU-GLU 替换常用的 SwiGLU。SwiGLU 两个分支都没有严格上界，异常大值相乘后可能继续放大。SiTU-GLU 用平滑的双曲正切上限压住极端值，在常用区间仍保持相近形状。

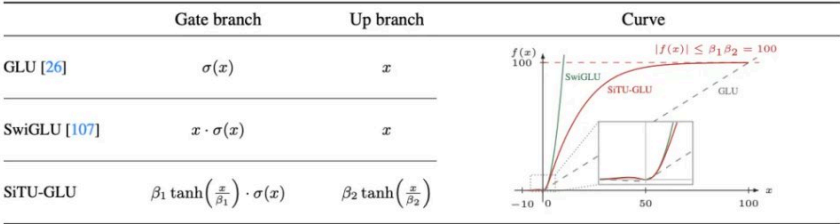


Figure 4: Gate and up branches of GLU, SwiGLU, and SiTU-GLU, together with their scalar responses, where σ denotes the sigmoid function. Both branches receive the scalar input x , and all curves share the domain $x \in [-10, 100]$; the inset magnifies the near-origin region. SiTU-GLU, shown in red with $\beta_1 = 4$ and $\beta_2 = 25$, closely follows SwiGLU near the origin and approaches the bound $|f(x)| \leq \beta_1 \beta_2 = 100$ for large positive inputs, whereas SwiGLU remains unbounded.

可以把它理解成给专业生产线安装限压阀。正常压力下阀门不影响工作，压力突然冲高时才介入，避免一次异常把后续设备全部顶坏。

KDA 的衰减下限也同时服务于训练稳定和 GPU 效率。Per-Head Muon，即**按注意力头分别执行 Muon 更新**，则避免梯度尺度较大的 Attention Head 主导整个投影矩阵。

不同注意力头分别归一更新，更像让不同车道各自校准方向，不能让一条最拥挤的车道决定整条道路怎么转弯。

这些稳定性改动未必直接贡献一个显眼的“1.x 倍”，却决定 Scaling Law 曲线能否在大规模训练中兑现。一个架构在小模型上更省 FLOPs，到了 2.8T 却频繁出现异常值，它的理论效率没有实际价值。

所以，K3 的 2.5 倍 Scaling Efficiency 应该被看作整套系统的训练总账。KDA、AttnRes 和 Stable LatentMoE 核心改善模型怎样计算，其他的底座 Infra 则保证硬件能持续交付这些 FLOPs。

算的动、传得动、算得满与学得快，最后会在这里汇合。

3T级模型，不再是魔法

回头看DeepSeek-V3、V4和Kimi K3，它们的技术路径存在大量重叠。

- 1) 用MoE扩大总参数，同时控制单Token激活成本。
- 2) 用MLA、稀疏注意力或者线性注意力降低长上下文成本。
- 3) 用低精度训练减少显存和通信压力。通过专家负载均衡、通信计算重叠和专用Kernel提高GPU利用率
- 4) 模型变深以后，再改造残差流、优化器和数值稳定机制，提高每单位FLOPs换来的模型质量。

具体招式可以不同，但往大里训练的方向已经相当清楚。

从研究上看，它没有单纯把K2继续放大，而是抓住了模型扩大以后最关键的三条信息通道。KDA改造Token与历史Token之间的信息流，让百万上下文不必承受全局注意力的平方成本。



AttnRes改造不同网络层之间的信息流，让93层网络不再只是不断累加和稀释历史表示。

LatentMoE则压缩Token与专家之间的信息流，使每Token激活专家数量翻倍后，单层通信量不必同步翻倍。

这三项设计表现出了非常好的研究审美。不是无限增加复杂度，而是准确找到系统中最危险的乘法项，再用克制的结构修改把它消掉。

但要让近千个专家在数千张GPU上连续运行，让通信、显存、调度和数值稳定不在任何一个局部崩掉，仍然需要极强的工程能力。

Kimi靠的是 FlashKDA、KDA上下文并行、MoonEP、Quantile Balancing、统一激活管理器和负载感知调度器，共同把架构上的理论收益兑现成真实训练吞吐。

研究负责找到方向，Infra负责让方向跑起来。

研究品味，工程能力两手抓，K3则证明了中国实验室同样有能力重新设计前沿模型。

本文来源：[腾讯科技](#)

风险提示及免责条款

市场有风险，投资需谨慎。本文不构成个人投资建议，也未考虑到个别用户特殊的投资目标、财务状况或需要。用户应考虑本文中的任何意见、观点或结论是否符合其特定状况。据此投资，责任自负。

 40  收藏

分享到:  

写评论

请发表您的评论

 表情  图片

发表评论